

ISIMA

Rapport Big Data

TP 1, TP2, TP3

Quentin DETRÉ
13/01/2026



Table des matières

I.	TP 1 : MongoDB.....	1
A.	Principes mis en place	1
1.	Résumé.....	1
2.	Pour aller plus loin.....	1
B.	Problèmes rencontrés.....	3
II.	TP 2 : Redis	3
A.	Résumé	3
B.	Implémentation.....	3
C.	Problèmes rencontrés.....	4
III.	TP 3 : ElasticSearch	5
A.	Résumé	5
B.	Implémentation.....	5
C.	Problèmes rencontrés.....	7
IV.	Utilisation de l'IA dans le développement	7

I. TP 1 : MongoDB

A. Principes mis en place

1. Résumé

Dans le TP 1, on nous demandait de mettre en place l'environnement d'une application PHP, incorporant plusieurs services comme MongoDB, Redis, K6, Grafana ou ElasticSearch, le tout dans plusieurs conteneurs Docker.

Il fallait aussi importer les données publiques OpenData de l'inventaire de la bibliothèque de Clermont-Ferrand, dans la base de données MongoDB qu'on nous demandait alors d'initialiser.

Enfin, on devait réaliser un CRUD (Create, Read, Update, Delete), afin de permettre à un utilisateur sur le site d'ajouter un livre à la base de données, de consulter les détails d'un livre, de modifier un livre existant, et de supprimer un livre de la base de données.

2. Pour aller plus loin

Dans la section « Pour aller plus loin » du TP, il nous était demandé de réaliser une pagination sur la page index de l'outil. Pour ce faire, j'ai choisi d'implémenter un système de « flèches » dans la vue, qui permettent de naviguer soit vers la page précédente ou vers la page suivante, mais aussi vers la première page et vers la dernière page.

Après avoir testé ceci, je me suis rendu compte qu'il était difficilement réaliste de demander à l'utilisateur de naviguer comme ceci parmi 350 et quelques pages. J'ai donc réfléchi à un système qui permettrait d'entrer directement le numéro de la page souhaité et d'y naviguer directement.

68c1ea114b423cd3348d1f5	1908. Voyage de Catalogne	Georges Desdunes du Désert	voir / éditer / supprimer
-------------------------	---------------------------	----------------------------	---------------------------

Aussi, c'est à cette étape que j'ai implémenté le système de « recherche », qui filtre par titre et par auteur avec une chaîne de caractère saisie par l'utilisateur.

Rechercher		Titre ou auteur	Rechercher	
Reference	Titre		Auteur	Actions
68c1ea114b423cd3348cf7a			Laflamme, Louis	voir / éditer / supprimer

On va alors définir 3 paramètres dans le code : le numéro de la page actuelle, la limite d'éléments sur une même page, et le mot-clé de recherche.

```

16 // Paramètres
17 $page = isset($_GET['page']) ? (int)$_GET['page'] : 1;
18 $search = isset($_GET['search']) ? $_GET['search'] : '';
19 $limit = 10;

```

On compte ensuite combien d'éléments il y a au total dans la base de données MongoDB. Ensuite, on vérifie que l'utilisateur n'essaye pas d'accéder à une page qui n'existe pas, en divisant par la limite d'éléments sur une page par le nombre d'éléments dans la BDD.

```
107 // Pagination MongoDB
108 $totalDocuments = $collection->countDocuments($filter);
109 $maxPages = ceil( num: $totalDocuments / $limit);
110 if ($page < 1) $page = 1;
111 if ($page > $maxPages && $maxPages > 0) $page = $maxPages;
112 $skip = ($page - 1) * $limit;
```

On filtre aussi la recherche avec une Regex si la recherche n'est pas vide. On y définit qu'on peut rechercher la chaîne de caractère parmi les titres ou parmi les auteurs.

```
95 // Filtre
96 $filter = [];
97 if (!empty($search)) {
98     $regex = new Regex($search, flags: 'i');
99     $filter = [
100         '$or' => [
101             ['titre' => $regex],
102             ['auteur' => $regex]
103         ]
104     ];
105 }
```

Enfin, on fait la requête MongoDB pour récupérer seulement les éléments qui nous intéressent : on utilise un « offset » (*numéro de page - 1*) * *limite du nombre d'éléments sur une page*. Cela permet de récupérer le bon nombre d'éléments au bon endroit de la liste d'éléments dans la BDD. La requête MongoDB utilise alors les fonctions « skip » et « limit » pour sauter dans la liste du bon nombre d'éléments, et ensuite limiter le nombre de résultats retournés.

```
// Récupération
$cursor = $collection->find($filter, [
    'limit' => $limit,
    'skip' => $skip,
    'sort' => ['titre' => 1]
]);
```

B. Problèmes rencontrés

Le TP 1 s'est déroulé sans problèmes majeurs. Il y a eu quelques soucis concernant l'installation de MongoDB sur Windows. Aussi, beaucoup de lenteurs sur mon PC quand je travaillais ont été constatées, essentiellement dues à mon environnement Windows non optimisé pour utiliser Docker et PHP. Dans l'ensemble, j'étais déjà familier avec les environnements PHP et les CRUD, donc c'était un travail que j'ai trouvé plutôt facile.

II. TP 2 : Redis

A. Résumé

L'utilisation de Redis dans ce projet s'est faite dans le cadre de l'implémentation d'un système de « Cache ». Avec le fonctionnement type « clé-valeur », il est facile d'implémenter ce genre de systèmes.

On devait donc initialiser un environnement Redis en utilisant la librairie Predis, et le SGBD Redis Commander, ainsi que de modifier les données du .env pour permettre la connexion, et activer ou désactiver le système de cache (REDIS_ENABLE).

B. Implémentation

J'ai donc implémenté la solution de cache. La création des clés pour les données en cache est comme suit : « livres:search=" . md5(\$search) . ":page=" . \$page ». Par la suite, si l'utilisateur effectue une même recherche qu'il a déjà effectué, l'outil devrait retrouver les résultats en retrouvant la clé et la page.

```
21 // Clé de cache
22 $cacheKey = "livres:search=" . md5($search) . ":page=" . $page;
```

```
29 // Logique data
30 if ($redis && $redis->exists($cacheKey)) {
31     // On lit depuis Redis
32     $data = json_decode($redis->get($cacheKey), associative: true);
33     $list = $data['list'];
34     $maxPages = $data['maxPages'];
35     $sourceType = 'Redis (Cache)';
36     $fromCache = true;
```

Si la clé correspondant à ce résultat de recherche n'existe pas, on exécute la suite de la requête à MongoDB, sinon on se contente du résultat de Redis. Si la requête MongoDB s'exécute, il faut alors la stocker en cache pour la prochaine fois que l'utilisateur fera la requête.

```
131 // Mise en cache
132 if ($redis) {
133     $dataToCache = [
134         'list' => $list,
135         'maxPages' => $maxPages
136     ];
137     // Stockage pour 60 secondes
138     $redis->set($cacheKey, json_encode($dataToCache));
139     $redis->expire($cacheKey, seconds: 60);
```

J'ai choisi de stocker le résultat de la requête pour seulement 1 minute pour plusieurs raisons. La première est que ce cache est seulement pour rendre la navigation plus fluide en cas de rafraîchissement ou de navigation rapide entre les pages. La deuxième est qu'en utilisant ce système de cache, si un utilisateur B modifie une ligne que l'utilisateur A a dans son cache, ce dernier ne verra pas le changement. Aussi, c'est pourquoi il est important d'invalider le cache après chaque modification de donnée (Create, Update ou Delete).

```
32 // Invalidation du cache Redis
33 $redis = getRedisClient();
34 if ($redis) {
35     $keys = $redis->keys(pattern: 'livres:');
36     if (!empty($keys)) {
37         foreach ($keys as $key) {
38             $redis->del($key);
39         }
40     }
41 }
```

C. Problèmes rencontrés

Les problèmes majeurs rencontrés à cette étape étaient plus de l'ordre de la confusion que de réels problèmes techniques. J'ai eu des soucis de connexion avec le cache Redis, principalement car les noms que j'utilisais dans mon .env n'étaient pas les bons.

Aussi, un autre souci que j'ai eu était lors des tests avec K6 et Grafana. Je n'explique toujours pas pourquoi dans le détail, mais l'application est nettement plus lente en utilisant le cache Redis qu'en utilisant une requête MongoDB. Ma théorie est qu'étant sous Windows, le sous-système Linux qui fait tourner mes conteneurs Docker ralenti dramatiquement la requête Redis. Mais ce n'est qu'une théorie.

III. TP 3 : ElasticSearch

A. Résumé

Ce TP consistait en l'implémentation d'ElasticSearch, afin d'indexer les données pour accélérer la recherche.

B. Implémentation

Il a d'abord fallu créer une fonction « indexation.php » pour permettre de générer le fichier à partir de tous les éléments de la base de données. Tout d'abord, on supprime les indexations préexistantes afin de nettoyer avant de créer un nouvel index.

```
17 // Suppression de l'ancien index
18 try {
19     $es->indices()->delete(['index' => 'books']);
20     echo "Ancien index supprimé.\n";
21 } catch (\Exception $e) {
22     echo "L'index n'existait pas encore.\n";
23 }
```

Ensuite, on récupère les données de la base de données MongoDB, et on ajoute deux variables, « total » et « errors ». Total représentera le nombre d'éléments indexés avec succès, et errors le nombre d'éléments qui n'ont pas pu être indexés.

```
25 // Lecture des données
26 $cursor = $collection->find();
27 $total = 0;
28 $errors = 0;
```

Puis, on va nettoyer les données de la base de données, afin de ne pas indexer des données nulles, et de permettre aussi de rechercher des auteurs « Anonymes » et des « Titres inconnu ».

```
32 foreach ($cursor as $document) {
33     $docArray = (array)$document;
34     // --- NETTOYAGE DES DONNEES ---
35     // On force la conversion en string
36     $id = (string)$docArray['_id'];
37     //Titre : S'il est vide on met un placeholder
38     $titre = !empty($docArray['titre']) ? $docArray['titre'] : 'Titre inconnu';
39     // Auteur : S'il est vide on met "Anonyme"
40     $auteur = !empty($docArray['auteur']) ? $docArray['auteur'] : 'Anonyme';
41     // Siècle : On s'assure que c'est un nombre ou 0
42     $siecle = isset($docArray['siecle']) ? (int)$docArray['siecle'] : 0;
```

On va enfin indexer chaque élément sous cette forme JSON.

```

44     $params = [
45         'index' => 'books',
46         'id'    => $id,
47         'body'  => [
48             'titre' => $titre,
49             'auteur' => $auteur,
50             'siecle' => $siecle
51         ]
52     ];

```

Et avec un « try / catch », on vérifie si l'élément qu'on traite a pu être indexé ou s'il a été en erreur.

```

try {
    $es->index($params);
    $total++;
    if ($total % 200 == 0) echo "$total livres traités...\n";
} catch (Exception $e) {
    $errors++;
    echo "Erreur sur l'ID $id : " . $e->getMessage() . "\n";
}

```

Enfin, dans app.php, on vérifie si le champ de recherche n'est pas vide. S'il n'est pas vide, on recherche en utilisant l'index précédemment créé avec une la variable « params » sous la forme suivante :

```

43     // Cas de recherche avec index (ElasticSearch)
44     if (!empty($search)) {
45         try {
46             $sourceType = 'ElasticSearch';
47             $params = [
48                 'index' => 'books',
49                 'body'  => [
50                     'from' => ($page - 1) * $limit,
51                     'size' => $limit,
52                     'query' => [
53                         'multi_match' => [
54                             'query' => $search,
55                             'fields' => ['titre^2', 'auteur'],
56                             'fuzziness' => 'AUTO'
57                         ]
58                     ]
59                 ]
60             ];

```



```
61 $results = $es->search($params);  
62 $totalDocuments = $results['hits']['total']['value'];  
63 $idsFromSearch = [];  
64 foreach ($results['hits']['hits'] as $hit) {  
65     $idsFromSearch[] = new ObjectId($hit['_id']);  
66 }
```

C. Problèmes rencontrés

Un problème que j'ai rencontré très tôt dans le développement a été résolu alors que je travaillais sur le TP 3. PhpStorm ne gardait pas les fenêtres de code ouvertes, et à chaque fois que je cliquais en dehors de la fenêtre de code, par exemple dans la console PhpStorm, ou bien sur mon navigateur pour tester le site, tous les fichiers de code que j'avais ouvert se fermaient tout seul. Il s'avère que PhpStorm essayait de synchroniser le code que j'avais ouvert avec un sous-système Linux en arrière-plan de mon OS Windows, et qu'il faisait time out ces synchronisations à chaque clic.

IV. Utilisation de l'IA dans le développement

Afin de pouvoir réaliser ce projet, j'ai utilisé à plusieurs moments des outils d'IA générative. Premièrement, j'ai utilisé Google Gemini pour essayer de trouver des solutions à certains problèmes qui me prenaient trop de temps à résoudre par moi-même. Typiquement, le souci de synchronisation de PhpStorm a été solutionné grâce à l'IA, et je n'aurai probablement pas pu arriver à une solution moi-même. En effet, après de longues minutes de recherche, je suis arrivé à la conclusion que cette erreur ne semble pas documentée sur internet, ou alors dans une langue que je ne maîtrise pas.

Aussi, Gemini m'a aidé pour réaliser plusieurs fichiers de code, notamment « load-test.js » et « indexation.php ». N'ayant jamais utilisé ni K6, ni Elasticsearch, j'ai fait améliorer mon code existant afin d'avoir quelque chose de plus complet, qui respecte un peu mieux les normes de développement propres à ces outils.

De plus, j'ai chargé Gemini de faire une « Code Review » de mon fichier app.php. En effet, vous pourrez voir qu'il est relativement long, et que le code qui s'y trouve n'est pas vraiment intuitif. N'ayant pas fait de PHP depuis longtemps, j'ai jugé bon de demander son avis à Gemini pour éviter les éventuels bugs, ainsi que d'éliminer le code superflu.

Enfin, j'ai utilisé l'IA « Scribens » pour corriger les éventuelles fautes d'orthographe qui auraient pu se glisser dans ce rapport.